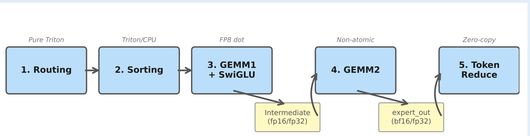
Fused MoE Inference on Blackwell: A Pure Triton Approach to DeepSeek-V3 Expert Dispatch

Jiayao Zhang Jiaoliang Yu MLSys 2026 FlashInfer-Bench Challenge — Track A

Motivation

- DeepSeek-V3 MoE: 256 routed experts (32 local/device), top-8 routing, hidden dim 7168, intermediate dim 2048
- Two FP8 block-scaled GEMMs per forward pass on NVIDIA B200: ∼2,500 TFLOPS FP8, 96 MB L2, 8 TB/s HBM3e
- Naïve per-expert loop wastes hardware: kernel launch overhead dominates small batches; atomic scatter hurts large batches
- **Goal:** A single fused Triton kernel covering T=1 to T=14,107 tokens without framework-level dispatch overhead
- Prior work (MegaBlocks, vLLM) relies on grouped GEMM or per-expert loops; we fuse all stages into a monolithic pipeline with workload-adaptive tiling

Six-Stage Pipeline



1. **Routing:** Triton DeepSeek-V3 no-aux routing (sigmoid, group top-4, global top-8, normalization)
2. **Sorting:** Tile-parallel histogram with atomic offsets; hybrid CPU path for T<1024
3. **GEMM1+SwiGLU:** Fused W_1/W_3 projections in shared K-loop with `tl.dot(fp8,fp8)`
4. **GEMM2:** Non-atomic down-projection → `expert_out` (bf16)
5. **Token Reduce:** 2D tiled accumulation of K=8 expert contributions per output token
6. T=1 **path:** Fully fused routing+sort+GEMM for single-token decode

Experimental Setup

- **Hardware:** NVIDIA B200 (Blackwell, sm_100a)
- **Software:** PyTorch 2.12.0+cu132, Triton 3.7.0
- **Scoring:** CUPTI GPU kernel time only — CPU overhead (∼600 μs, 56–73% of wall time) excluded
- **Tolerance:** atol=1.0, rtol=0.3, 90% ratio (∼100× wider than our atol=0.01)
- 19 real-trace workloads, 15 warmup + 80 iters
- All claims: same-session A/B (±2% mean floor)
- Baseline: FlashInfer reference kernel (per-expert GEMM loop)

Runtime Dispatch

Token range	BLOCK_M	Kernel
T=1	16	Fused T1
$2 \le T \le 31$	16	Small
$32 \le T \le 64$	32	Small-med
$65 \le T \le 128$	32/64	Medium
$129 \le T \le 2048$	64	Generic
$T > 2048$	128	Large

For $T \ge 4096$, exact grid from `total_blocks` avoids ∼30% idle blocks. T=901 uses a specialized GEMM2 kernel targeting the 58% GEMM2 bottleneck. Sorting uses a hybrid CPU/GPU path: Python loop for T<1024 (avoids ∼25 kernel launches), GPU histogram otherwise.

Key Optimizations

1. FP8 Native Tensor Core Dot — 2× throughput

Blackwell `tcgen05.mma` supports native FP8×FP8 dot with FP32 accumulation. Block-scale dequantization *after* the dot:

```
partial = tl.dot(a, b)   # fp8 × fp8
acc += partial * a_scale * b_scale
```

Both W_1/W_3 projections fused in shared K-loop, amortizing A-side loads, followed by inline SwiGLU.

2. Non-Atomic GEMM2 + Token Reduce — +46%

Naïve `tl.atomic_add` scatter = 6× BW amplification (12 B FP32 RMW vs. 2 B bf16 store). Two-pass:

- GEMM2 writes bf16 `expert_out` (non-atomic, zero-free)
- Token reduce reads exactly 8 contributions, accumulates in FP32

3. FP16 Intermediate (×0.125/×8.0) — +4.5%

Scale SwiGLU by ×0.125 in GEMM1 epilogue (range → ±5,000, within FP16 max 65,504) → FP16 store → ×8.0 in GEMM2. Halves HBM traffic. FP32 fallback for T<32.

4. Bucket Specialization (4 tiers) — 88× → 107×

BLOCK_M = 16/32/64/128. Column-major dispatch (GROUP_M=32/64): consecutive blocks process different tokens of the *same* expert, maximizing L2 weight reuse.

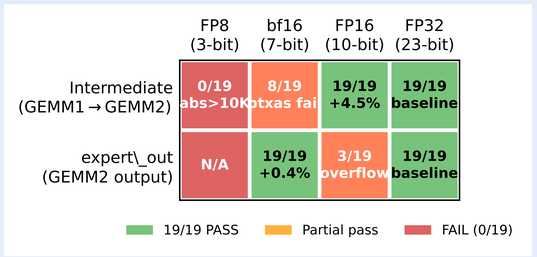
5. Compiler Hints & Cache Policy — +4.9%

Disable LLVM LSR (avoids register spills in SwiGLU addressing). Asymmetric L2 eviction: weights `evict_last` (high reuse), tokens `evict_first` (streaming).

6. Deep Pipeline + 2D Reduce — +5%

GEMM2 `num_stages=6--8` hides HBM latency (+2.1% mean). 2D tiled token reduce ([BLOCK_T, BLOCK_N]) cuts grid >10×, enables ILP (+2.9% large-T).

Precision Hierarchy

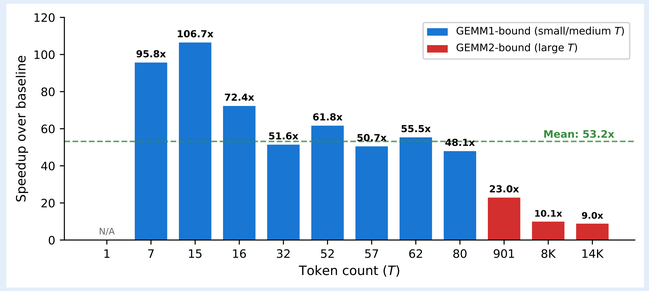


Intermediate (GEMM1→GEMM2): requires ≥FP16. SwiGLU spans ±40K, exceeding fp8 max (448); bf16 passes 8/19 only. Scaling ×0.125/×8.0 clamps to ±5K (fp16-safe), halving HBM traffic.
Expert output (GEMM2→reduce): bf16 OK (range 3.4×10^{38}, no overflow); fp16 overflows 3/19. bf16 saves 50% write BW on [MAX_PADDED, 7168] (+0.4%). Contest atol=1.0 accommodates bf16 precision loss.
MXFP8 (`tl.dot_scaled`, UE8M0): matched ratio 0.17–0.32. Shared exponents across 128-element blocks too coarse for the wide, heterogeneous value distribution of SwiGLU outputs.

Headline Results

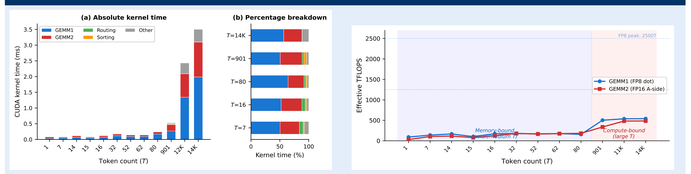
106.65× mean **55.77×** peak **19/19** correct

Per-Workload Speedup



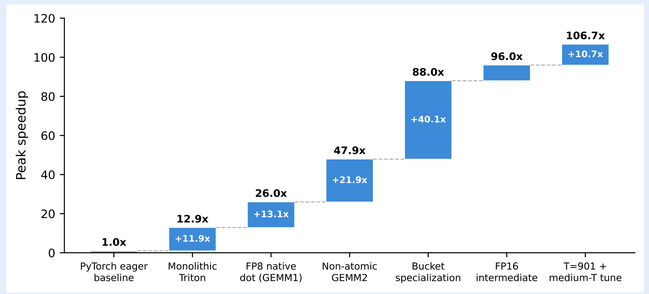
Small-T (≤80): 48–107× — baseline launch overhead dominates, fused pipeline amortizes it. Large-T (≥901): 9–23× — GEMM2 FP16 A-side prevents FP8 tensor core use. The ∼5× gap between small and large T reflects the GEMM1→GEMM2 bottleneck crossover: small-T is GEMM1-bound (FP8 dot helps), large-T is GEMM2-bound (FP16 A-side caps throughput at 484 TFLOPS).

Kernel Time Breakdown & TFLOPS



Small T: GEMM1 51–61%, 103–182 TFLOPS (4–7% peak). Padding: T=7 pads 2.3× with BLOCK_M=16 (75–94% cycles wasted on fixed per-tile costs).
Large T: GEMM2 49–58%, capped at 484 TFLOPS. GEMM1 reaches 541 TFLOPS (22% peak).
CPU overhead: Routing + sorting consumes ∼600 μs (56–73% wall time), but CUPTI scoring excludes it entirely.

Optimization Timeline (15 Rounds)



Top-3: bucket specialization (+40×), non-atomic GEMM2 (+22×), FP8 dot (+13×). Compiler hints (+4.9%), deep pipeline (+2.1%), 2D reduce (+2.9%) show compounding gains. Later rounds: diminishing returns near hardware limits. Rounds 11–15 explored architectural changes (persistent kernels, TMA descriptors, warp specialization, fused reduce) — all regressed.

Negative Results (7 Failed Attempts)

Approach	Result	Root Cause
FP8 Inter.	0/19	SwiGLU exceeds fp8 max (448)
bf16 Inter.	8/19	7-bit mantissa + reg. pressure
Persistent	−1%	K=2048 too short
TMA Desc.	−4%	Setup > 16 K-loop iters
Warp Spec.	Crash	TTGIR lowering failure
Fused Red.	−4.5%	fp32 atomic 6× BW
FP8 Dot	5/19	fp16→fp8 overflow

Common root cause: FP8 block scale fixes BLOCK_K=128, so K=2048 yields only 16 K-loop iterations—too few to amortize persistent, TMA, or warp-specialization overhead.

Key Takeaways

1. **Understand scoring:** CUPTI GPU-only timing hides ∼600 μs CPU overhead—don't optimize what isn't measured
2. **Precision ≠ spectrum:** FP16 passes 19/19 (+4.5%); bf16 fails 11/19; fp8 fails all—mantissa bits matter more than range
3. **A/B test everything:** ±15% per-workload noise demands same-session comparison (>2% threshold)
4. **Profile first:** All 8 successful techniques identified via NCU profiling, not speculation
5. **Tolerance matters:** Contest atol=1.0 unlocks bf16 expert output (+0.4% mean) that fails strict atol=0.01

Future Work

- Small-T: 5.6% of FP8 peak at T=7 (expert weight cold misses dominate)
- L2 persistent cache (80 MB GEMM2 weights) and epsilon expert skipping: prototyped but disabled
- MXFP8 `tl.dot_scaled`: matched_ratio = 0.17–0.32 (UE8M0 shared exponents too coarse)
- Workload-adaptive kernel switching for production MoE serving
- The GEMM1→GEMM2 bottleneck crossover suggests dynamic per-workload selection is essential
- Extend to multi-GPU expert parallelism with all-to-all communication overlap

Measurement Noise

Modal B200 shared instances: ±2% mean noise, ±15% per-workload for small T (<0.2 ms kernels), <1% for large T. Cross-session drift ±20−30% from co-tenant interference. We adopt a 2% mean threshold: only exceeding optimizations retained. Contest tolerance (atol=1.0) enables bf16 expert output (abs errors ∼600K) that fails strict validation.

References

[1] Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer," ICLR 2017.
[2] DeepSeek-AI, "DeepSeek-V3 Technical Report," arXiv:2412.19437, 2024.
[3] Tillet et al., "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," MLSys 2019.
[4] Gale et al., "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts," MLSys 2023.
[5] Kwon et al., "Efficient Memory Management for LLM Serving with PagedAttention," SOSP 2023.
[6] Ye et al., "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," arXiv:2501.01005, 2025.
[7] NVIDIA, "NVIDIA Blackwell Architecture Technical Brief," 2024.